

JOURNAL 3

Frontend Implementation

BlurGuard – Real-Time Screen-Share Sensitive Data Censor

Project	BlurGuard – Real-Time Screen-Share Sensitive Data Censor
Development Stage	Frontend Prototype Implementation
Technology Used	React, Vite, React Router, JavaScript, CSS
State Management	React Context API
Student	Lakshita

1. Objective

The objective of this development phase was to implement the frontend prototype of the BlurGuard – Real-Time Screen-Share Sensitive Data Censor system using React. The frontend was developed to provide an interactive interface for users to configure privacy settings, select the screen or application to monitor, and visualize real-time detection and protection of sensitive information during screen sharing. The work focused on application setup, navigation, user interaction flows, real-time monitoring interface, configuration settings, and frontend integration of core features to demonstrate the major workflow before backend and model integration.

2. Development Environment and Technologies

The frontend was developed using the following technologies:

- **React** – used for developing reusable user interface components.
- **Vite** – used for creating and running the React development project.
- **JavaScript** – used for frontend application logic.
- **React Router** – used for navigation between different application pages.
- **CSS** – used for styling dashboards, forms, cards, navigation bars, buttons, and status indicators.
- **React Context API** – used for maintaining shared application state between frontend components.

3. Frontend Project Setup

The frontend project was organized as a modular React application. Separate components and page sections were considered for authentication, dashboard monitoring, privacy settings, detection results, and session controls. This structure allows individual parts of the interface to be developed and modified without affecting the complete application.

The major frontend sections include:

- Login / User Session
- Main Dashboard
- Screen and Application Selection
- Privacy Rules and Settings
- Live Monitoring
- Detection and Risk Results
- Protected Region / Censor Controls
- Alerts and User Review

4. Application Navigation

Frontend navigation was structured so that the user can move between the major BlurGuard functions from a single application interface. The planned major routes and views are:

- Login
- Dashboard
- Screen / Window Selection
- Privacy Settings
- Monitoring Session
- Detection Results
- Alerts / Review
- Session History / Logs

The navigation structure keeps the frontend organized and provides a clear path from starting a session to monitoring and protecting sensitive information.

5. Login and User Session Interface

A simple login interface was included as the entry point of the BlurGuard frontend. The interface is intended to establish the user session before the monitoring dashboard is opened.

The interface contains:

- Email / username field
- Password field
- Login button
- Session entry and logout controls

At the prototype stage, the interface focuses on frontend interaction. Actual authentication, authorization, and secure session handling can be integrated with the backend in a later stage.

6. BlurGuard Dashboard

The main dashboard was designed as the central control interface for a screen-protection session. It provides the user with a quick overview of the current monitoring state and the actions required to start or manage protection.

The dashboard includes:

- Monitoring status
- Selected screen or application
- Protection status
- Number of detected sensitive regions
- Current risk level
- Start / Pause / Stop monitoring controls
- Access to privacy settings
- Recent alerts or detection events

7. Screen / Application Selection

A dedicated interface was designed for selecting the screen, application window, or display region that BlurGuard should monitor. This prevents unnecessary monitoring of unrelated content and provides the user with control over the monitoring scope.

The selection interface can display available screens or windows and allow the user to confirm the target before starting a monitoring session.

8. Privacy Rules and Settings

A privacy settings interface was designed to allow the user to configure which categories of information should be protected. The interface can provide controls for sensitive data categories and protection

behaviour.

Example configuration options include:

- Email addresses
- Phone numbers
- Passwords and authentication secrets
- API keys and access tokens
- Payment or card-related information
- Personal identification information
- QR codes or other sensitive visual content
- Blur, pixelate, or redact as the protection method
- Detection confidence threshold

9. Real-Time Monitoring Interface

The monitoring interface was designed to represent the live state of the BlurGuard system while a screen-sharing session is active. The interface provides visual feedback so that the user can understand whether monitoring and protection are running.

The monitoring view can display:

- Current monitoring state
- Selected screen or application
- Frames being monitored
- Detection activity
- Protected regions
- Current risk level
- Alerts requiring user attention

10. Detection Results and Alert Interface

A detection-results section was designed to show information returned by the detection and analysis layer. The purpose of this interface is to make the protection process understandable to the user without exposing the sensitive information itself unnecessarily.

The interface can represent:

- Detection category
- Confidence level
- Risk level
- Detected region
- Protection action applied
- Time of detection

An alert or review action can be provided when the system detects content that requires confirmation or when the confidence level is below the configured threshold.

11. Frontend Component Structure

The frontend was organized around reusable components so that common interface elements can be shared across different pages. A modular component structure also makes future integration with backend APIs and real-time detection services easier.

Component / Page	Purpose
Login	User session entry
Dashboard	Central monitoring controls and status
ScreenSelector	Select screen or application to monitor
PrivacySettings	Configure sensitive-data rules
MonitoringPanel	Display live monitoring state
DetectionPanel	Display detection and risk information
AlertPanel	Display alerts and review actions
ProtectionStatus	Show current censoring/protection state

12. Frontend State and Integration

Shared frontend state was considered for maintaining information such as the active monitoring session, selected screen or application, privacy settings, detection results, and protection status. React Context API can be used to make this information available to multiple components without maintaining separate copies of the same state.

The frontend is structured so that the detection and analysis engine can later provide real-time results through an API, WebSocket connection, or another application-level communication mechanism. Similarly, the protected output can later be connected to the screen-sharing integration layer.

13. Frontend Integration and Testing

The frontend prototype was checked from the perspective of navigation, interface interaction, form behaviour, and consistency of the monitoring workflow.

The main testing activities include:

- Starting the frontend application.
- Navigation between the main pages.
- Opening the monitoring dashboard.
- Selecting a screen or application.
- Changing privacy settings.
- Starting and stopping the monitoring interface.
- Displaying detection and risk information.
- Displaying protection status and alerts.

These checks help ensure that the frontend provides a consistent user flow before connecting the interface to the complete detection and screen-sharing pipeline.

14. Current Frontend Result

The frontend stage establishes the user-facing structure of BlurGuard. The main dashboard, screen/application selection, privacy configuration, monitoring controls, detection results, and alert interfaces have been defined as the primary frontend workflow.

The current frontend prototype provides the foundation for integrating the actual OCR, PII/secret detection, computer-vision analysis, protection engine, and screen-sharing output in subsequent development stages.

15. Current Limitations

The current frontend stage has the following limitations:

- Backend API integration is not yet part of this development stage.
- Actual authentication and authorization require backend support.

- Real-time detection results depend on integration with the detection engine.
- Actual screen capture and protected output require system-level integration.
- Persistent storage and complete session history require backend/database integration.

16. Learning Outcomes

The frontend implementation provided practical understanding of:

- React component-based development.
- Frontend routing and multi-page navigation.
- Designing interfaces for real-time monitoring systems.
- Interactive forms and configuration controls.
- Shared state management using React Context API.
- Modular frontend architecture.
- Frontend integration planning for backend and real-time services.
- Debugging and testing an interactive prototype before full system integration.

17. Result

The following development activities were completed for this journal:

Activity	Status
Frontend project structure defined	Completed
Main dashboard interface designed	Completed
Screen / application selection interface	Completed
Privacy settings interface	Completed
Monitoring interface	Completed
Detection and alert interface	Completed
Frontend component structure	Completed
Frontend workflow testing	Completed

18. Conclusion

Journal 03 documents the frontend implementation stage of the BlurGuard – Real-Time Screen-Share Sensitive Data Censor project. The frontend structure establishes the main user-facing workflow for starting a monitoring session, selecting a screen or application, configuring privacy rules, viewing monitoring and detection information, and controlling the protection process.

The completed frontend design provides a foundation for the next development stages, including integration with the OCR and sensitive-data detection engine, context and risk analysis, sensitive-region tracking, protection engine, and screen-sharing output.